

API with Key

In this mini project you will create a project similar to MP2. However, this time, you must choose an API that requires an API key/secret. And you must avoid exposing your key/secret to the client. So, you will need to establish some communication between the code running in the browser and the code running on the server.

Demo

Here is a demo we have created using the following API:

<https://www.visualcrossing.com/weather-data-editions>. And the following is an example of what your finished mini project may look like: <https://cs391-weather-app.vercel.app/>

Task 1. Initialize a new Next.js project

1. Follow the process described in discussion or lecture slides. You may use Tailwind CSS but are not required to.

Task 2. Obtain an API key/secret

1. Choose an API that requires a key/secret and returns JSON. Please do **NOT** choose one that requires you to input payment information. You are allowed to use the same API as our demo project.
 - a. The following are some APIs you may choose from. However, you are NOT limited to this list. Our only request is that you do not choose an API that requires inputting a payment method
 - i. <https://github.com/harvardartmuseums/api-docs>
 - ii. <https://docs.ksoft.si/api/lyrics-api>
 - iii. <https://developer.iconfinder.com/reference/overview>
 - iv. You can find more at the following link. Choose one labeled "apiKey": <https://github.com/public-apis/public-apis>
2. Follow the instructions/documentation for the API your chosen to obtain a key/secret
3. Create a file called `.env.local` at the root of your Next.js project and paste your unique API key. Name it appropriately for your chosen API. It should look something like this:

```
WEATHER_API_KEY="your-key-here"
```

4. This key is only accessible through server-side code. By default, Next.js code is server-side unless it is called from or a child of a client-side function (this can be confusing. ask questions on Piazza or office hours). Here is an example of how you can access it.

```
const WEATHER_API_KEY = process.env.WEATHER_API_KEY;
```

Task 3. Figure it out and have fun!

1. Start with an outline / the things you know how to do, such as creating the necessary pages. Then, create the component(s) to display the data you will receive from the API and test it using mock data
2. Figure out how to call your chosen API. Different APIs require the key to be sent differently. The following are some common ways: as a query parameter, in the request body, in the headers. Most well-documented APIs will explain what they expect and will likely include code-snippets
 - a. If you need help with this, ask on Piazza and include a link to the documentation for the API you've chosen
3. Decide how you want to send the data to the frontend (see below) and implement it

Potential Approaches

The following are explanations of some ways you can implement some of the functionality requirements. Note that this documentation is NOT exhaustive and you will benefit significantly from doing your own research.

1. Backend-Frontend communication: This can be implemented by rendering the forecast server-side or by sending the forecast data, likely as JSON, to the client and rendering it client-side. JSON forecast data can be sent from the server to the client via an API endpoint that you create or a server-side function. If you implement the awaiting of a server-side function, check the network tools of your browser.
 - a. Server-side rendering
 - i. Await the forecast from the API server-side, then return a display of this information. This is my preferred method (the TA)
 - ii. It may be helpful to think of the weather forecast as data you already have. Think about how you can display that. After that works, think about live fetching data from the API and displaying the real forecast.
 - b. JSON API
 - i. This is the most "React" way of doing this. But not necessarily the most efficient.
 - ii. You will need to create an api route/endpoint that takes in a city name and returns the forecast for that city as JSON.
 - iii. The client will need to await a fetch request to this endpoint, as we have done several weeks during discussion. You would effectively just be wrapping the API endpoint with your own code.
 - iv. <https://nextjs.org/docs/pages/api-reference/functions/next-response#json>

- v. <https://nextjs.org/docs/pages/api-reference/functions/next-request>
- c. Server-side function
 - i. This is a very "Next.js" way for doing this and does not have an equivalent on most other frameworks. Ultimately, this is similar to the JSON API method, but it is simpler for the developer to implement and understand.
 - ii. Create an asynchronous server-side function that retrieves data from the API and returns it. Ensure this is a server-side function to prevent leaking your API key to the client. Hint: **"use server"**
 - iii. This function can be used in a server-side route or it can be called, similar to the **fetch()** function, in a client-side component.

Functionality Requirements:

1. Your API key must NOT be exposed to the client
2. You must display some data from the API
3. Proper error handling must be implemented in the case that you exceed your daily API-call limit or if the API is down. Don't show a loading screen forever.
4. There must be at least two pages/routes.
5. You do not need to worry about a malicious actor exploiting your endpoint. ie, it is okay if you simply wrap your chosen API.

In addition to these functionality requirements, your project must also adhere to programming best practices (e.g. DRY, separation of concerns) and be styled appropriately.

The **only** external packages you may use are those used for UI/styling, such as MUI and styled-components.

You may **NOT** use the Next.js pages router; you should not have a directory called **"pages"** or use a **getServerSideProps()** function.